Vibe Coding Workshop

Build Data Products with AI-Assisted Development

77 Accelerator Skills | **10 AppKit Skills** | **9 Pipeline Stages**

Powered by Databricks + AI Coding Assistants

Agenda

Act I — Welcome & Context

Vibe coding, repo structure, skill anatomy

Act II — Data Product Accelerator

Animated journey through 9 stages, 77 accelerator skills

Act III — Platform Training

Databricks Apps, AppKit, Lakebase, SDP, Genie

Act IV — AppKit Workshop Journey

Branch-aware AppKit lifecycle with skills

Act V — Putting It All Together

Live demo prompts, resources, Q&A

~70 slides | Export: HTML (transitions) | PDF (handout) | PPTX (shareable)

What is Vibe Coding?

> **AI-assisted development** where you collaborate with AI tools to rapidly build, iterate, and deploy production-quality data products.

You Describe

- Business requirements (PRD)
- Schema CSVs, data sources
- "Build a Gold layer for this schema"

AI Implements

- Reads agent skills for best practices
- Generates production-grade code
- Follows enterprise patterns automatically

The secret sauce: Agent Skills — structured `SKILL.md` files that encode best practices and patterns the AI reads before writing any code.

Repository Overview

This is a **monorepo** with four components:

Key rule: Framework directories are **read-only inputs**. Generated code goes to **repo root**.

Two Pathways

Start Here → Path A: Build an App or Path B: Build a Data Pipeline

Path A — Databricks App `apps_lakebase/`

1. Scaffold AppKit project
2. Build UI from PRD (mock data)
3. Deploy to Databricks Apps
4. Wire Lakebase backend
5. Deploy + E2E test

10 skills, branch-aware lifecycle

Path B — Data Pipeline `data_product_accelerator/`

1. Gold Design from schema CSV
2. Bronze tables + test data
3. Silver DLT pipelines
4. Gold implementation
5. Semantic → Observability → ML → GenAI

77 skills, 9 stages

Both paths work together — build the pipeline first, then deploy an app on top, or vice versa.

What is a SKILL.md?

A **structured instruction file** that any AI coding assistant can read and follow.

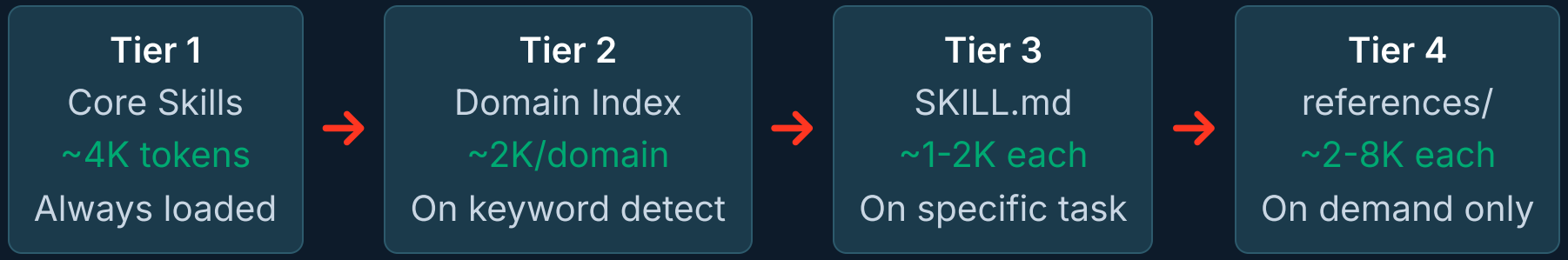
```yaml
---
name: gold-layer-design
metadata:
  role: orchestrator
  pipeline_stage: 1
  workers:
    - design-workers/01-grain-definition
    - design-workers/02-dimension-patterns
  common_dependencies:
    - databricks-expert-agent
    - naming-tagging-standards
---
# Gold Layer Design Orchestrator
## When to Use This Skill
- Designing a Gold layer from scratch
- Creating dimensional models
```

Anatomy

- **YAML front matter** — metadata, dependencies, pipeline stage
- **When to Use** — routing keywords
- **Mandatory Dependencies** — skills to read first
- **Step-by-step instructions** — the actual guide
- `references/` — detailed patterns (loaded on demand)
- `scripts/` — validation utilities

Progressive Disclosure: Tiered Loading

The framework uses **4 tiers** to stay within the AI's context budget:



Zone	Token Budget	Strategy
Green (0-20K)	Load freely	Multiple SKILL.md files + 2-3 references
Yellow (20-50K)	Be selective	SKILL.md files are fine, pick references carefully
Red (50K+)	Minimize	Reference paths only, run scripts as black boxes

Orchestrator vs Worker Pattern

Orchestrator `00-*`

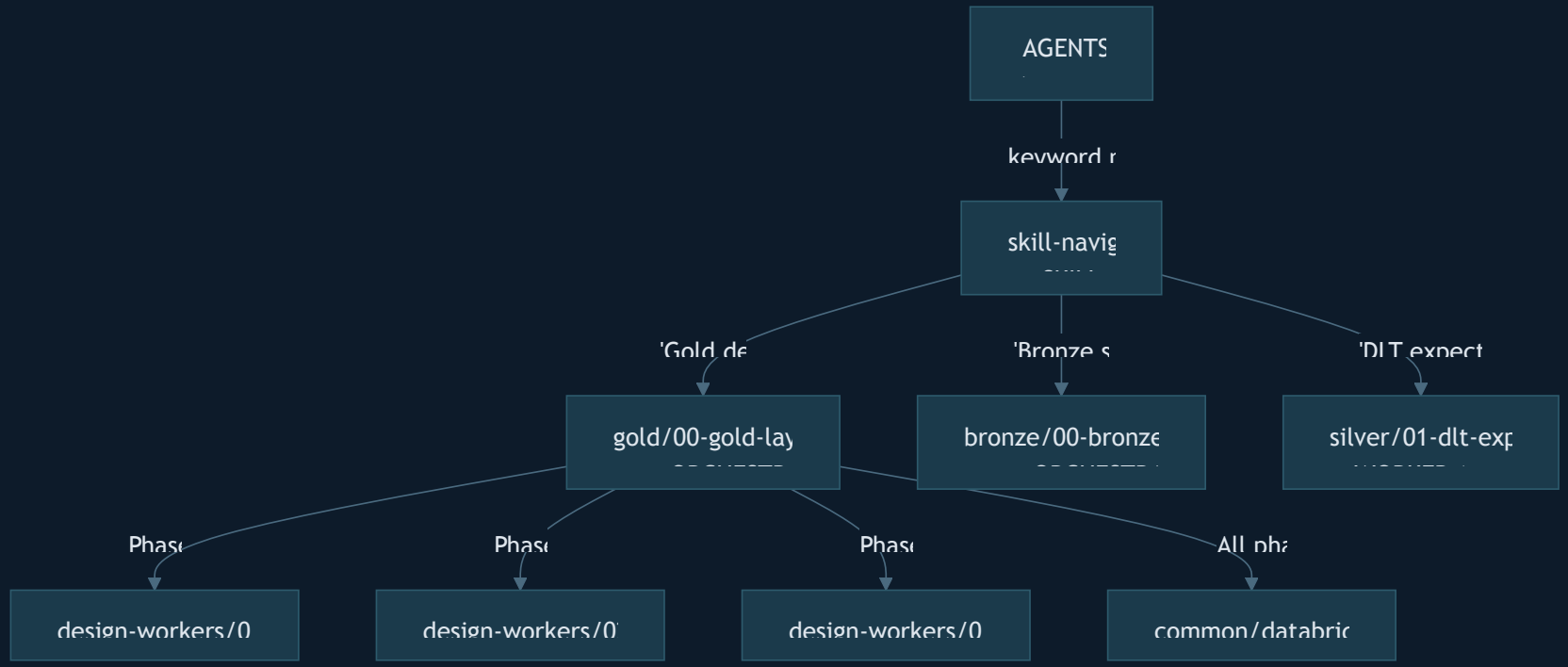
- Manages the **end-to-end workflow** for a stage
- Calls worker skills via mandatory Read pattern
- Defines phases, dependencies, deliverables
- **Entry point** for "build X from scratch"

Worker `01-, 02-, ...`

- Handles a **specific pattern** within a stage
- Can be used standalone or called by

```
gold/
├── 00-gold-layer-design/    ← ORCHESTRATOR
│   ├── SKILL.md
│   └── references/
├── design-workers/
│   ├── 01-grain-definition/
│   ├── 02-dimension-patterns/
│   ├── 03-fact-table-patterns/
│   ├── 04-conformed-dimensions/
│   ├── 05-erd-diagrams/
│   ├── 06-table-documentation/
│   └── 07-design-validation/
└── pipeline-workers/
    ├── 01-yaml-table-setup/
    ├── 02-merge-patterns/
    ├── 03-deduplication/
    ├── 04-grain-validation/
    └── 05-schema-validation/
```

How the AI Agent Navigates



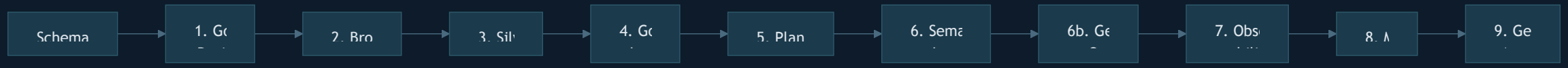
Routing algorithm: Match keywords → prefer orchestrator → orchestrator calls workers per phase → common skills loaded as dependencies.

Act II

The Data Product Accelerator

9 stages — 77 skills — one animated journey

The 9-Stage Design-First Pipeline



Design-First means: Design the Gold model (target) first, then build the layers that feed it.

Input	Output
`data_product_accelerator/context/*.csv`	`gold_layer_design/` , `src/` , `plans/` , `resources/` , `databricks.yml`

Stage 1: Gold Layer Design

Orchestrator: `gold/00-gold-layer-design` ENTRY POINT

What It Does

- Reads schema CSV from `context/`
- Applies dimensional modeling methodology
- Generates ERDs, YAML schemas, documentation

Key Deliverables

- `gold_layer_design/yaml/*.yaml` — table definitions
- `gold_layer_design/erd_master.md`

Non-Negotiable Defaults

Every Gold YAML schema **must** include:

```yaml
clustering: auto
table_properties:
  delta.enableChangeDataFeed: "true"
  delta.enableRowTracking: "true"
  delta.autoOptimize.optimizeWrite: "true"
  delta.autoOptimize.autoCompact: "true"
  layer: "gold"
```

Stage 1: Gold Design — Worker Tree

The orchestrator calls **7 design workers** across its phases:

```
gold/00-gold-layer-design  ← ORCHESTRATOR (reads schema CSV)
│
├── Phase 2: Dimensional Modeling
│   ├── design-workers/01-grain-definition      grain types, decision tree
│   ├── design-workers/02-dimension-patterns    role-playing, junk, degenerate, SCD
│   ├── design-workers/03-fact-table-patterns   measure additivity, factless, snapshots
│   └── design-workers/04-conformed-dimensions  bus matrix, drill-across queries
│
├── Phase 3: ERD Creation
│   └── design-workers/05-erd-diagrams          master/domain/summary based on table count
│
├── Phase 4: Documentation
│   └── design-workers/06-table-documentation   business onboarding guide, lineage CSV
│
└── Phase 8: Validation
    └── design-workers/07-design-validation     YAML ↔ ERD ↔ Lineage cross-check
```

Stage 2: Bronze Layer

Orchestrator: `bronze/00-bronze-layer-setup` ORCHESTRATOR

Philosophy

Bronze is optimized for **testing, demos, and rapid prototyping** — not production ingestion.

What It Creates

- Table DDLs with `CLUSTER BY AUTO`
- Faker-generated realistic test data
- Asset Bundle job YAML (Serverless)

Worker

- `bronze/01-faker-data-generation`

Non-Negotiable Defaults

```
CREATE TABLE IF NOT EXISTS {catalog}.{schema}.{table}
(...)
USING DELTA
CLUSTER BY AUTO
TBLPROPERTIES (
  'delta.enableChangeDataFeed' = 'true',
  'delta.autoOptimize.optimizeWrite' = 'true',
  'delta.autoOptimize.autoCompact' = 'true',
  'layer' = 'bronze'
)
```

Every job uses **Serverless + Environments V4** + `notebook_task` with `base_parameters`.

Stage 3: Silver Layer

Orchestrator: `silver/00-silver-layer-setup` ORCHESTRATOR

What It Creates

1. `dq_rules` **Delta table** — centralized rules in Unity Catalog
2. `dq_rules_loader.py` — pure Python module to load rules
3. **Silver SDP/DLT notebooks** — with expectations from Delta table
4. **Pipeline YAML** — Serverless, ADVANCED edition, Photon
5. **DQ monitoring views** — per-table

Key Pattern: Delta Table DQ Rules

```python
# Rules stored in Delta table, not hardcoded
rules = load_rules("silver_bookings", "critical")

@dlt.table(
    table_properties={
        "delta.enableRowTracking": "true"
    },
    cluster_by_auto=True
)
@dlt.expect_all_or_drop(rules)
def silver_bookings():
    return read_stream("bronze_bookings")
```

Benefits: Runtime-updateable, auditable

Stage 4: Gold Implementation

Orchestrator: `gold/01-gold-layer-setup` ORCHESTRATOR

Takes the YAML schemas from Stage 1 and **builds the actual tables, merge scripts, and constraints**.

```
gold/01-gold-layer-setup  ← ORCHESTRATOR (reads YAML schemas)
│
├── pipeline-workers/01-yaml-table-setup     YAML-driven CREATE TABLE DDLs
├── pipeline-workers/02-merge-patterns       MERGE INTO with SCD Type 2
├── pipeline-workers/03-deduplication        ROW_NUMBER dedup strategies
├── pipeline-workers/04-grain-validation     Verify fact table grain integrity
└── pipeline-workers/05-schema-validation    Cross-check DDL vs YAML schema
```

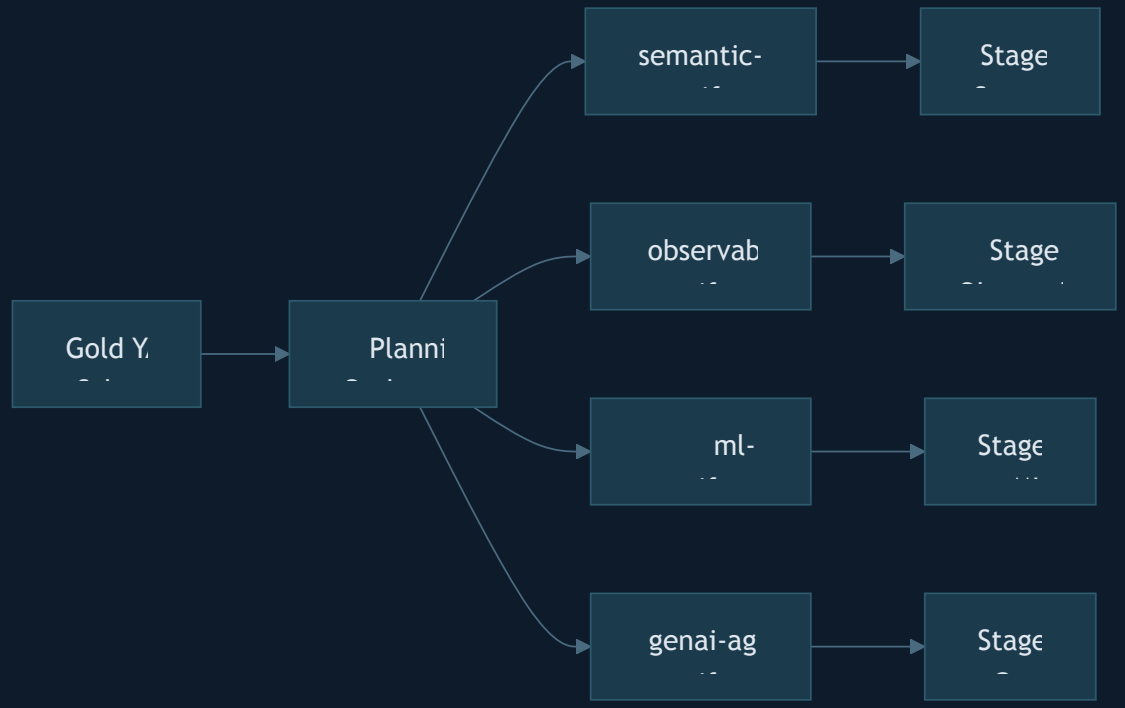
Design → Implementation Handoff

Stage 1 produces **YAML schemas** (the contract). Stage 4 consumes them to generate DDL, merge notebooks, and validation scripts. The YAML is the single

Stage 5: Planning

Orchestrator: `planning/00-project-planning` ORCHESTRATOR

The planning stage generates **YAML manifest files** — contracts that downstream stages consume.



Stage 6: Semantic Layer

Orchestrator: `semantic-layer/00-semantic-layer-setup` ORCHESTRATOR

What It Creates

1. **Metric Views** — YAML semantic definitions
2. **Table-Valued Functions (TVFs)** — parameterized SQL for Genie
3. **Genie Spaces** — NL analytics with instructions + benchmarks

Workers (4)

- `01-metric-views-patterns`
- `02-databricks-table-valued-`

Data Asset Priority for Genie

```
Priority 1: Metric Views
  → Pre-defined business metrics
  → Genie uses these FIRST

Priority 2: TVFs
  → Parameterized queries
  → Genie passes parameters

Priority 3: Raw Tables
  → Full flexibility
  → Genie writes custom SQL
```

Core principle: Business context drives AI quality. The richer your Metric Views

Stage 6b: Genie Optimization

Standalone Orchestrator: `semantic-layer/05-genie-optimization-orchestrator`

An **autonomous optimization loop** with 4 specialized workers:

```
05-genie-optimization-orchestrator  ← STANDALONE ORCHESTRATOR
│
├── genie-optimization-workers/01-genie-benchmark-generator
│   Generate benchmark question sets with expected SQL
│
├── genie-optimization-workers/02-genie-benchmark-evaluator
│   Run benchmarks, score accuracy & repeatability
│
├── genie-optimization-workers/03-genie-metadata-optimizer
│   Optimize table/column comments, agent instructions
│
└── genie-optimization-workers/04-genie-optimization-applier
    Apply optimizations via Genie API, re-benchmark
```

Stage 7: Observability

Orchestrator: `monitoring/00-observability-setup` ORCHESTRATOR

Workers (4)

Worker	What It Does
`01-lakehouse-monitoring`	Table monitors, row counts, schema drift
`02-aibi-dashboards`	Lakeview dashboards for data health
`03-sql-alerting`	SQL-based alerts with notifications
`04-anomaly-detection`	Freshness, completeness, stale table detection

What It Creates

- Lakehouse Monitors on Gold tables
- AI/BI Lakeview dashboards
- SQL alerts with email/Slack notifications
- Anomaly detection for freshness & completeness
- Asset Bundle job YAML for scheduled monitoring

Auto-triggered: Anomaly detection can

Stage 8: ML Pipelines

Orchestrator: `ml/00-ml-pipeline-setup` ORCHESTRATOR

What It Creates

- MLflow experiments and tracking
- Feature engineering notebooks
- Model training with hyperparameter tuning
- Model registry integration
- Batch inference pipelines
- Asset Bundle job YAML

Key Patterns

- **MLflow Tracking** — all experiments logged
- **Unity Catalog Models** — registered in UC for governance
- **Feature Tables** — reusable feature engineering
- **Serverless Jobs** — training and inference
- **A/B Testing** — model comparison

Stage 9: GenAI Agents

Orchestrator: `genai-agents/00-course-orchestrator` ORCHESTRATOR

The current GenAI course is a routed progression: foundation, Track A custom Agent Apps, AppKit 2-Apps wiring, and the MLflow SDLC.

```
genai-agents/00-course-orchestrator  ← ORCHESTRATOR
│
├── foundation/          UC resources, MLflow, tracing, tools, KA, AI Gateway
├── tracks/A-custom-agent-apps/  Python Agent App on Databricks Apps
├── apps_lakebase/skills/06d   AppKit ↔ Agent App OBO proxy
├── apps_lakebase/skills/07-08 Chat history + feedback
└── sdlc/                Prompt registry, evals, registration, deploy, monitor
```

From a custom Agent App to a rich AppKit frontend, with evaluation, prompt versioning, feedback, deployment, and production monitoring.

Cross-Cutting: 8 Common Skills

These shared skills apply across **all 9 stages**:

Skill	What It Provides	Used By
databricks-expert-agent	Core SA behavior, "Extract Don't Generate" principle	All stages
naming-tagging-standards	`snake_case`, COMMENTs, PII tags, budget policies	All DDL
databricks-asset-bundles	Job/pipeline YAML, Serverless config, Environments V4	Stages 2-9
databricks-autonomous-operations	Deploy → Poll → Diagnose → Fix → Redeploy loop	Any deployment
databricks-python-imports	Pure Python modules, `sys.path` patterns	Silver, Gold, ML
databricks-table-properties	TBLPROPERTIES: CDF, Row Tracking, Auto-Optimize	Bronze-Gold
schema-management-patterns	`CREATE SCHEMA IF NOT EXISTS` with governance	Bronze-Gold
unity-catalog-constraints	PRIMARY KEY, FOREIGN KEY syntax, surrogate keys	Silver-Gold

Minimum read: Always load `databricks-expert-agent` + `naming-tagging-standards`.

The Complete Skill Tree — 77 Accelerator Skills

```
skills/
├── skill-navigator/                navigator
├── admin/ (4 utility skills)

├── gold/
│   ├── 00-gold-layer-design        S1
│   ├── 01-gold-layer-setup         S4
│   ├── design-workers/ (7)         S1
│   └── pipeline-workers/ (5)       S4

├── bronze/
│   ├── 00-bronze-layer-setup       S2
│   └── 01-faker-data-generation    S2

├── silver/
│   ├── 00-silver-layer-setup       S3
│   ├── 01-dlt-expectations         S3
│   └── 02-dqx-patterns             S3

├── planning/
│   └── 00-project-planning         S5

├── semantic-layer/
│   ├── 00-semantic-layer-setup     S6
│   ├── workers (4)                 S6
│   ├── 05-genie-optimization       S6b
│   └── genie-opt-workers/ (4)      S6b

├── monitoring/
│   ├── 00-observability-setup      S7
│   └── workers (4)                 S7

├── ml/
│   └── 00-ml-pipeline-setup        S8

├── genai-agents/
│   ├── 00-course-orchestrator      S9
│   ├── foundation/                 S9
│   ├── tracks/                     S9
│   ├── sdlc/                       S9
│   └── capstone/                   S9

├── common/ (8 shared skills)
└── exploration/ (1 utility)
```

Orchestrator | Worker | Meta/Common

Routing Algorithm

How the skill-navigator matches your request to the right skill:

```
1. User request received
2. Detect domain keywords (see routing table)
3. IF keyword matches an ORCHESTRATOR skill
   → Route to orchestrator
   → Orchestrator calls workers via mandatory Read pattern
4. IF keyword matches a WORKER skill AND no orchestrator context
   → Route to worker directly (standalone mode)
5. IF keyword matches a WORKER skill AND orchestrator is active
   → Let orchestrator handle it (don't load worker separately)
6. IF ambiguous or multi-step
   → Ask user to clarify, or follow lifecycle order
```

Example: "Create Silver DLT pipeline with expectations"
→ Matches "Silver" + "DLT" → Routes to `silver/00-silver-layer-setup` (orchestrator)
→ Orchestrator loads `01-dlt-expectations-patterns` at the right phase

Act III

Platform Training

The building blocks: Databricks Apps, AppKit, Lakebase, SDP, Genie, Unity Catalog

Module 1: Databricks Apps

What is Databricks Apps?

Managed hosting for full-stack web applications that run inside the Databricks platform.

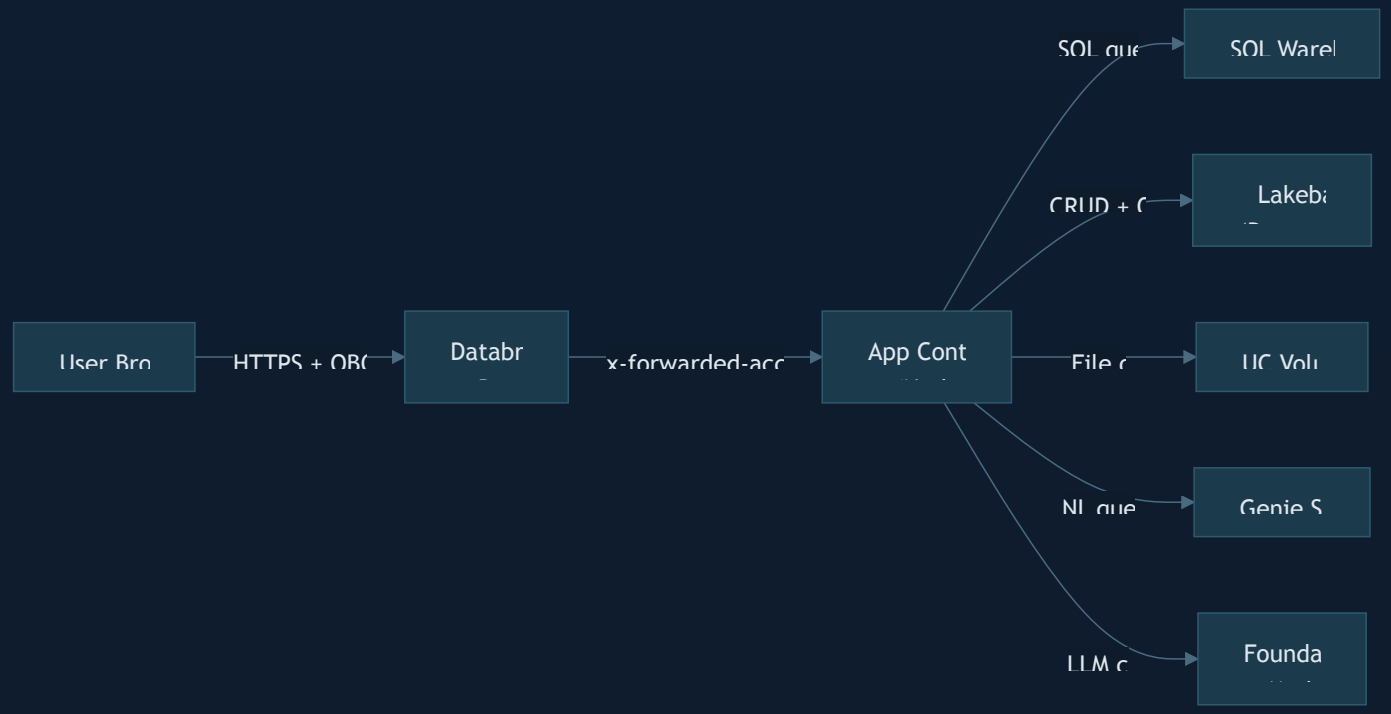
Key Properties

- **Node.js runtime** — your app runs in a container
- **Service Principal identity** — app gets its own SP for API access
- **OBO (On-Behalf-Of)** — user identity flows via `x-forwarded-access-token`
- **Ephemeral filesystem** — no persistent local storage

Platform Constraints

Constraint	Value
Startup timeout	10 minutes
HTTP proxy timeout	120 seconds
Max apps per workspace	100
Max file size in bundle	10 MB
Persistent storage	None (use Lakebase or UC Volumes)

Databricks Apps Architecture



Runtime Environment Variables (auto-injected)

Variable	Description
`DATABRICKS_HOST`	Workspace URL
`DATABRICKS_APP_PORT`	Port to bind (default: 8000)
`DATABRICKS_APP_NAME`	App name in Databricks

Module 2: AppKit — TypeScript SDK

What is AppKit?

A **TypeScript full-stack framework** with a plugin architecture for building Databricks Apps.

Two Packages

`@databricks/appkit` — Backend

- Express server with plugin system
- SQL query execution with caching
- Type generation from SQL files
- Telemetry and health checks

`@databricks/appkit-ui` — Frontend

- React hooks (`useAnalyticsQuery` , `useGenieChat`)

The Data Flow

```
client/src/App.tsx
  → useAnalyticsQuery("key", params)
    → POST /api/analytics/query/key
      → server/server.ts
        → config/queries/key.sql
          → SQL Warehouse
            → JSON response
              → React renders
```

All type-safe end-to-end via `npm run typegen` .

AppKit Project Structure

After scaffolding with `databricks apps init`:

```
my-app/
├── server/
│   ├── server.ts          ← backend entry point
│   └── .env               ← local dev env vars
├── client/
│   ├── index.html
│   ├── vite.config.ts
│   └── src/
│       ├── main.tsx
│       ├── App.tsx        ← main React component
│       └── appKitTypes.d.ts ← auto-generated
├── config/queries/
│   └── *.sql              ← SQL files = query keys
├── tests/smoke.spec.ts
├── app.yaml               ← Databricks Apps config
├── package.json
└── databricks.yml         ← bundle config
```

Key Patterns

Plugin registration in `server/server.ts`:

```ts
import { createApp, server, analytics,
         lakebase } from "@databricks/appkit";

await createApp({
  plugins: [
    server(),       // Express server
    analytics(),    // SQL Warehouse queries
    lakebase(),     // PostgreSQL CRUD
  ],
});
```

Custom routes via `server.extend()`:

AppKit Dev Loop

The development workflow — in order, every time:

1. Write SQL → 2. npm run typegen → 3. Check types → 4. Build UI → 5. npm run dev → 6. Deploy

SQL Query File Convention

```sql
-- config/queries/spend_summary.sql
-- @param startDate DATE
-- @param endDate DATE
SELECT department, SUM(amount) as total
FROM catalog.schema.expenses
WHERE expense_date BETWEEN :startDate AND :endDate
GROUP BY department
```

- `.sql` → runs as **Service Principal** (shared cache)
- `.obo.sql` → runs as **user** (per-user cache)

Frontend Hook Usage

```tsx
import { useAnalyticsQuery }
  from "@databricks/appkit-ui/react";
import { sql }
  from "@databricks/appkit-ui/js";

function SpendTable() {
  const params = useMemo(() => ({
    startDate: sql.date("2025-01-01"),
    endDate: sql.date("2025-12-31"),
  }), []);

  const { data, loading, error } =
    useAnalyticsQuery("spend_summary", params);
  // ...
}
```

Module 3: Lakebase — Managed PostgreSQL

What is Lakebase?

Autoscaling managed PostgreSQL on Databricks with OAuth-based authentication.

Key Features

- **OAuth token rotation** — 1-hour tokens, auto-refreshed with 2-minute buffer
- **Scale-to-zero** — `suspend_timeout_duration` stops idle compute
- **Projects / Branches / Endpoints** — Git-like database management
- **No passwords** — all auth via Databricks OAuth

Configuration in AppKit

```ts
// server/server.ts
import { createApp, server, lakebase }
  from "@databricks/appkit";

await createApp({
  plugins: [server(), lakebase()],
});
```

```
# .env (local dev)
LAKEBASE_ENDPOINT=projects/<id>/branches/production/endpoints/primary
PGHOST=<endpoint-hostname>
PGPORT=5432
PGDATABASE=databricks_postgres
PGSSLMODE=require
```

The `lakebase()` plugin gives you

Lakebase Database Design

Principles for designing schemas in AppKit + Lakebase applications:

Design Process

1. Read the PRD → identify **entities** (nouns = tables)
2. Define **columns** (atomic: separate first/last name)
3. Choose **PKs** (`bigint generated always as identity`)
4. Establish **relationships** (FK for 1:N, junction for M:N)
5. **Normalize** (1NF → 2NF → 3NF)
6. Create **seed data**, test queries,

Anti-Patterns to Avoid

- `serial` → use `bigint generated always as identity`
- `varchar(255)` → use `text` (same performance)
- `float` for money → use `numeric(10,2)` (exact)
- `timestamp` → use `timestamptz` (timezone-aware)
- Comma-separated lists → use junction tables

Module 4: Spark Declarative Pipelines (SDP/DLT)

What is SDP?

Declarative ETL pipelines — formerly Delta Live Tables (DLT). Define WHAT your pipeline produces; the engine handles HOW.

Two APIs

Modern API (new projects):

```
from pyspark import pipelines as dp

@dp.table()
def silver_bookings():
    return spark.readStream("bronze_bookings")
```

Legacy API (still needed for expectations):

Pipeline Non-Negotiables

Setting	Value	Why
Serverless	`true`	No cluster management
Edition	`ADVANCED`	Required for expectations
Photon	`true`	Performance
Liquid Clustering	`AUTO`	Auto-optimized layout
Row Tracking	`true`	Downstream MV refresh

SDP/DLT: Delta Table DQ Rules

The key pattern: **DQ rules in a Delta table, not hardcoded.**

Traditional (Fragile)

```python
# Rules hardcoded in notebook
@dlt.expect_or_drop(
    "valid_amount",
    "amount > 0"
)
@dlt.expect(
    "valid_status",
    "status IN ('active','cancelled')"
)
def silver_orders():
    ...
```

Changing rules = code change + redeploy.

Delta Table (Portable)

```sql
-- dq_rules Delta table
INSERT INTO dq_rules VALUES
('silver_orders', 'valid_amount',
 'amount > 0', 'critical'),
('silver_orders', 'valid_status',
 'status IN ( ... )', 'warning');
```

```python
# Loader reads rules at runtime
rules = load_rules("silver_orders", "critical")

@dlt.table(cluster_by_auto=True)
@dlt.expect_all_or_drop(rules)
def silver_orders():
    return dlt.read_stream("bronze_orders")
```

Changing rules = `UPDATE` the table. No

Module 5: Genie Spaces

What is Genie?

AI/BI natural language query interface — ask questions in English, get SQL answers from your data.

Three Components

1. **Agent Instructions** (business context)
   - ≤20 lines of General Instructions
   - Domain terminology, business rules
   - "Revenue means SUM(amount) WHERE status='completed'"
2. **Data Assets** (what Genie can query)
   - Priority: Metric Views > TVFs > Tables

Core Principle

> **Business context drives AI quality.**
> The richer your instructions and metadata, the better Genie answers.

Quality Levers

Lever	Impact
Agent Instructions	Highest — teaches business rules
Table/Column COMMENTs	High — Genie reads them for SQL generation

Genie in AppKit

Embed Genie directly in your Databricks App:

Plugin Setup

```ts
// server/server.ts
import { createApp, server, genie }
  from "@databricks/appkit";

await createApp({
  plugins: [
    server(),
    genie({
      spaces: {
        sales: "01ABCDEF12345678",
        support: "01GHIJKL87654321",
      },
    }),
  ],
});
```

Multiple spaces via named aliases

React Component

```tsx
import { GenieChat }
  from "@databricks/appkit-ui/react";

function GeniePage() {
  return (
    <div style={{ height: 600 }}>
      <GenieChat alias="sales" />
    </div>
  );
}
```

Full-featured chat UI with:

- SSE streaming responses
- Conversation history & replay

Module 6: Unity Catalog & Volumes

Unity Catalog — Governance Layer

The backbone of data governance across all stages:

- **Catalogs → Schemas → Tables** — 3-level namespace
- **Constraints** — PRIMARY KEY, FOREIGN KEY (informational)
- **Tags** — PII classification, cost center, layer tags
- **COMMENTs** — on tables and columns (Genie reads these!)
- **Volumes** — managed file storage

Files Plugin — UC Volumes in AppKit

```
import { createApp, server, files }
  from "@databricks/appkit";

await createApp({
  plugins: [
    server(),
    files({
      maxUploadSize: 5_000_000_000,
    }),
  ],
});
```

```
# Auto-discovered from env vars:
DATABRICKS_VOLUME_UPLOADS=/Volumes/catalog/schema/uploads
DATABRICKS_VOLUME_EXPORTS=/Volumes/catalog/schema/exports
```

React components: `DirectoryList`, `FileBreadcrumb`, `FilePreviewPanel`

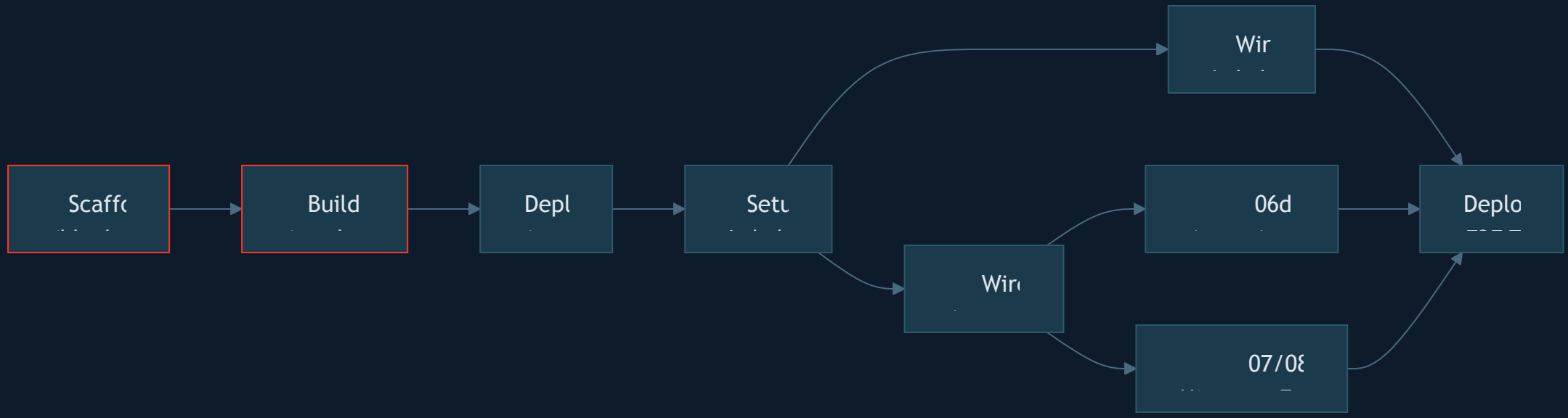
Act IV

AppKit + Lakebase Workshop

Branch-aware lifecycle — from scaffold to production with live data and optional agent chat

The AppKit Lifecycle

1 Scaffold + Build | 2 Deploy Mock | 3 Setup Lakebase | 4 Wire Branches | 5 Deploy + E2E



Phases 1-2: Functional UI with mock data (no database needed)

Phases 3-5: Add Lakebase and optional agent-chat branches, then verify live

Phase	Skills Used
1	`01-appkit-scaffold` + `02-appkit-build`

Phase 1: Scaffold

Skill: `01-appkit-scaffold`

Steps

1. Authenticate: `databricks auth login --host <URL>`
2. Derive app name from username + use case
3. Scaffold: `databricks apps init --template default`
4. Install dependencies: `npm install`
5. Verify: `npm run dev` → `http://localhost:8000`

What You Get

```
$APP_NAME/
├── server/server.ts      ← server()
├── client/src/App.tsx    ← your UI
├── config/queries/       ← SQL files
├── app.yaml
├── databricks.yml
└── package.json
```

Workshop Note

Scaffold a **blank** app (no plugins). Plugins are added later in Phase 4.

`TABLE_OR_VIEW_NOT_FOUND` errors from

Phase 1: Build from PRD

Skill: `02-appkit-build`

Process

1. **Read the PRD** — personas, journeys, data needs
2. **Design the UI** — screens, components, navigation
3. **Build with mock data** — static arrays in components
4. **Apply design quality** — distinctive, not generic

Mock Data Pattern

Design Quality Principles

Before coding, ask:

- What is the emotional tone?
- What makes this app feel *distinctive*?
- What is the single most important action?

Avoid "AI slop":

- Generic Inter font + purple gradients

Phase 2: Deploy with Mock Data

Skill: `03-appkit-deploy`

Deploy Sequence

```
# Build the app
npm run build

# Validate configuration
databricks apps validate --profile $PROFILE

# Deploy to Databricks Apps
databricks apps deploy --profile $PROFILE
```

Verify

```
# Check app status
databricks apps get $APP_NAME \
  --profile $PROFILE
```

Common Issues & Fixes

Error	Fix
Build fails	`npm run build` locally first
App won't start (timeout)	Check `app.yaml` command, review logs
403 on workspace	Verify profile with `databricks current-user me`
Max apps reached	Delete unused: `databricks apps delete <name>`

Key Rule

> **Do NOT improvise workarounds.**

Phase 3: Setup Lakebase Project

Uses `04-appkit-plugin-add` plus `apps_lakebase/prompts/03-setup-lakebase.md` to add the Lakebase package and bundle resources.

Steps

1. Install `@databricks/lakebase`
2. Add `postgres_projects` to `databricks.yml`
3. Add `valueFrom: postgres` and `DB_SCHEMA` to `app.yaml`
4. Validate app config
5. Leave `server.ts` unchanged until Phase 4

Bundle Resource Model

```
databricks.yml
└── resources.postgres_projects.my_db
    ├── project_id: $APP_NAME
    ├── pg_version: 17
    └── default_endpoint_settings

app.yaml
└── env:
    ├── LAKEBASE_ENDPOINT: valueFrom postgres
    └── DB_SCHEMA: $APP_NAME with hyphens replaced
```

Compute Sizing

Phase 4: Add Lakebase Plugin

Skill: `04-appkit-plugin-add`

Register the plugin in `server/server.ts`:

```ts
import { createApp, server, lakebase } from "@databricks/appkit";

await createApp({
  plugins: [server(), lakebase()],
});
```

Configure environment variables:

`.env` (local dev):

```
LAKEBASE_ENDPOINT=projects/<id>/branches/production/endpoints/primary
PGHOST=<endpoint-hostname>
PGPORT=5432
PGDATABASE=databricks_postgres
PGSSLMODE=require
DB_SCHEMA=app_data_schema
```

`app.yaml` (deployed):

```yaml
env:
  - name: LAKEBASE_ENDPOINT
    value: 'projects/<id>/branches/production/endpoints/primary'
  - name: PGHOST
    value: '<endpoint-hostname>'
```

Phase 4: Wire Lakebase Backend

Skill: `05-appkit-lakebase-wiring`

What Gets Built

1. **DDL** — `CREATE SCHEMA` + `CREATE TABLE IF NOT EXISTS`
2. **Seed data** — `INSERT ... ON CONFLICT DO NOTHING`
3. **API routes** — `server.extend()` for CRUD endpoints
4. **Response format** — `{ data: [...], source: "live"|"mock" }`

API Route Pattern

Mock Fallback Pattern

Every API route returns mock data if Lakebase is unavailable:

```
Database available?
  YES → { data: liveRows, source: "live" }
  NO  → { data: mockArray, source: "mock" }
```

This means the app **always works** — with or without a database.

Type Mapping (API Layer)

```
function mapBooking(row: any): Booking {
```

Phase 4: Frontend Hooks

`useLakebaseData` — Reusable Data Hook

```
function useLakebaseData<T>(endpoint: string) {
  const [data, setData] = useState<T[]>([]);
  const [source, setSource] = useState<"live" | "mock" | "loading">("loading");

  useEffect(() => {
    fetch(endpoint)
      .then(res => res.json())
      .then(json => { setData(json.data ?? []); setSource(json.source ?? "mock"); })
      .catch(() => setSource("mock"));
  }, [endpoint]);

  return { data, source };
}
```

`ConnectionStatus` — Visual Data Source Indicator

```
function ConnectionStatus({ source }: { source: "live" | "mock" | "loading" }) {
```

The Mock-to-Live Transition

Before (Mock Data)

```tsx
// Static arrays in components
const BOOKINGS = [
  { id: 1, guest: "Alice", ... },
  { id: 2, guest: "Bob", ... },
];

function BookingList() {
  return BOOKINGS.map(b =>
    <BookingCard key={b.id} booking={b} />
  );
}
```

● Mock Data — bookings

After (Live Data)

```tsx
// API-backed with mock fallback
function BookingList() {
  const { data, source } =
    useLakebaseData<Booking>("/api/bookings");

  return (
    <>
      <ConnectionStatus source={source} />
      {data.map(b =>
        <BookingCard key={b.id} booking={b} />
      )}
    </>
  );
}
```

● Live Data — bookings

The transition: replace `MOCK_ARRAY.map( ... )` with

Phase 5: Deploy + E2E Test

Skill: `03-appkit-deploy` (reused from Phase 2)

Deploy with Lakebase

```
# Build
npm run build

# Deploy (SP creates DB objects)
databricks apps deploy --profile $PROFILE
```

On first deploy with Lakebase, the **Service Principal** runs the DDL to create schemas, tables, and seed data.

E2E Verification

1. Open the deployed app URL

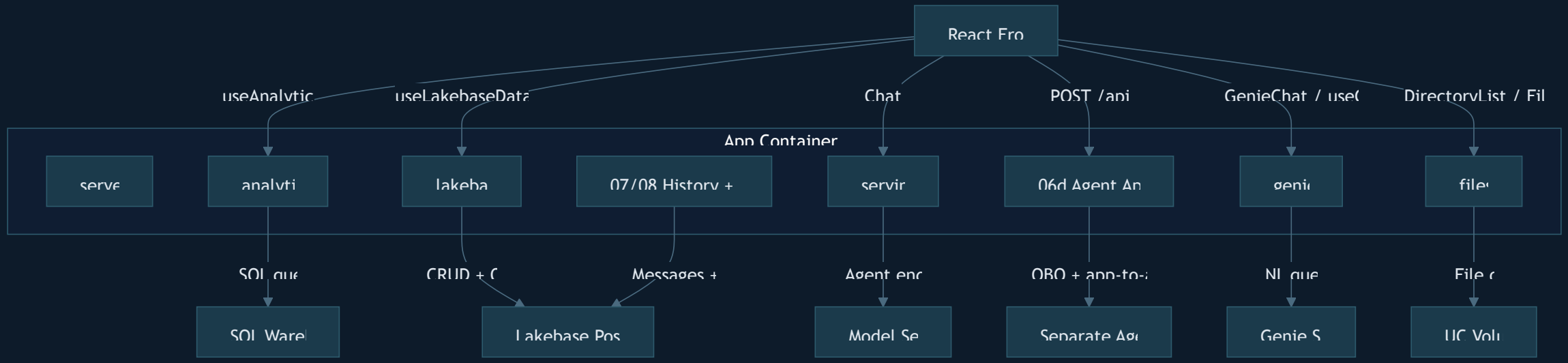
What Gets Verified

Check	Expected
App loads	No errors in console
ConnectionStatus	Green "Live Data"
Data displays	Real rows from Lakebase
Create record	New row appears
Update record	Changes persist
Delete record	Row removed
Idle 5+ min, reload	Reconnects to Lakebase

Troubleshooting

AppKit Plugin Architecture

AppKit plugins compose with custom proxy layers for agent-chat apps:



Each plugin or proxy layer is **independent** — add only what you need. The `server()` plugin is always required.

Skill Dependency Map

How the 10 AppKit skills connect to the branch-aware workshop lifecycle:

Phase → Skill Mapping

Phase	Primary Skill	Supporting
1. Scaffold	`01-appkit-scaffold`	`00-navigator`
1. Build	`02-appkit-build`	—
2. Deploy	`03-appkit-deploy`	—
3. Lakebase setup	`04-appkit-plugin-add`	`prompts/03-setup-lakebase.md`
4. Lakebase wiring	`05-appkit-lakebase-wiring`	—
4. Agent endpoint	`06-appkit-serving-wiring`	or `06d-appkit-agent-app-proxy`
4. Chat UX	`07-appkit-chat-`	`08-appkit-`

Skill Dependencies

```
00-appkit-navigator  ← routing only
│
01-appkit-scaffold      ← standalone
└── 02-appkit-build     ← needs scaffold
│
03-appkit-deploy        ← standalone
│   (used in Phase 2 AND Phase 5)
│
04-appkit-plugin-add    ← needs scaffold
└── 05-appkit-lakebase-wiring ← needs plugin
│
├── 06-appkit-serving-wiring ← Model Serving / Agent endp
├── 06d-appkit-agent-app-proxy ← separate Agent App
│
07-appkit-chat-history ← needs Lakebase + agent stream
└── 08-appkit-feedback ← needs chat trace IDs
```

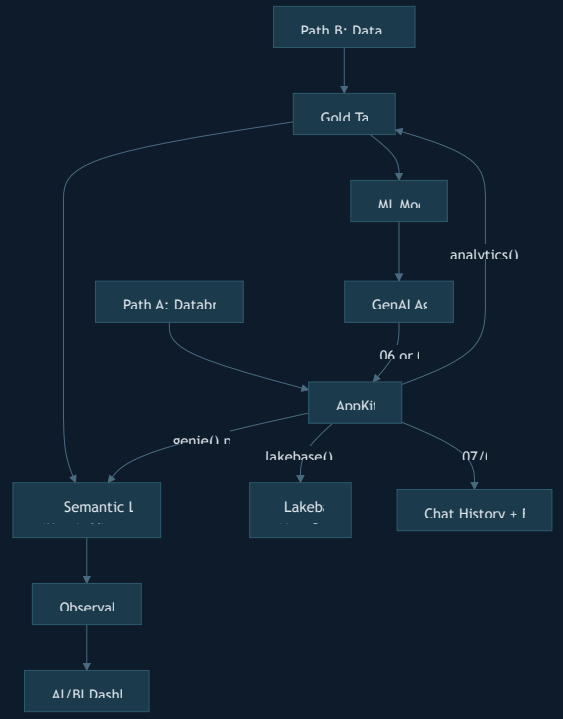
Cross-References to Training

Act V

Putting It All Together

Two paths, one destination — AI-powered data products on Databricks

Both Paths Converge



The complete picture:

- **Path B** builds the data foundation (Gold tables, semantic layer, ML models, agents)
- **Path A** builds the user-facing app that queries that foundation

Workshop Parameters

Fill in before starting. Referenced throughout all steps.

Parameter	Description	Your Value
`{workspace_url}`	Your Databricks workspace URL	`https://_____.cloud.databricks.com`
`{use_case_slug}`	Short app identifier (e.g., `bookings`)	_____
`{PROFILE}`	Databricks CLI profile name	_____
`{user_app_name}`	Lakebase project name (Phase 3 output)	_____
`{LAKEBASE_HOST}`	Lakebase endpoint hostname (Phase 3 output)	_____
`{agent_app_name}`	Optional separate Agent App name for 06d	_____
`{agent_app_url}`	Optional separate Agent App URL for 06d	_____

Pre-flight Check

```
# Verify workspace access
databricks current-user me --host {workspace_url}

# Verify CLI version (≥ 0.295.0)
```

Your Turn — Path A Prompts

Copy-paste these into your AI coding assistant to execute each phase:

Phase 1: Scaffold + Build

```
I want to build a Databricks App. My workspace is {workspace_url}.
Read @apps_lakebase/skills/01-appkit-scaffold/SKILL.md and scaffold a blank AppKit project.
Then read @apps_lakebase/skills/02-appkit-build/SKILL.md and build the UI from @docs/design_prd.md
using mock data arrays.
```

Phase 2: Deploy

```
Deploy the app to Databricks Apps. Read @apps_lakebase/skills/03-appkit-deploy/SKILL.md
App name: $APP_NAME, Profile: $PROFILE
```

Phase 3: Setup Lakebase

```
Set up Lakebase bundle resources for my AppKit app.
Read @apps_lakebase/skills/04-appkit-plugin-add/SKILL.md and follow
@apps_lakebase/prompts/03-setup-lakebase.md.
Do not modify server/server.ts in this step.
```

Your Turn — Path B Prompts

Stage 1: Gold Design

```
I have a customer schema at @data_product_accelerator/context/Wanderbricks_Schema.csv.
Please design the Gold layer using @data_product_accelerator/skills/gold/00-gold-layer-design/SKILL.md
```

Stage 2: Bronze

```
Create the Bronze layer with test data. Read @data_product_accelerator/skills/bronze/00-bronze-layer-setup/SKILL.md
Use the schema CSV at @data_product_accelerator/context/Wanderbricks_Schema.csv
```

Stage 3: Silver

```
Build the Silver DLT pipelines with DQ rules. Read @data_product_accelerator/skills/silver/00-silver-layer-setup/SKILL.md
```

One prompt per stage. One new conversation per stage.

See the full 9-stage guide for all prompts.

Resources & Links

Repository

- AGENTS.md — Root navigator
- QUICKSTART.md — 9-stage prompts
- Instructions.md — AppKit workshop guide
- PRE-REQUISITES.md — Setup checklist

AppKit Documentation

- AppKit Docs — Official docs
- `npx @databricks/appkit docs` — In-terminal docs
- Agent Skills Repo — Databricks

Platform Docs

- Databricks Apps
- Lakebase
- SDP/DLT
- Genie Spaces
- Unity Catalog

Framework

- Agent Skills Format — SKILL.md standard
- Skill Navigation Guide
- Skill Hierarchy Tree

Thank You

Build Something Amazing

77 Accelerator Skills + 10 AppKit Skills — ready to guide your AI assistant

Start now: Clone the repo, pick a path, paste a prompt

Questions?